

Downscale-State CNN

Klasyfikacja stanu downscalingu obrazów

Dataset RAISE · pipeline próbek 256x256 · klasy CTRL , x8 , x16 , x32

Cel projektu

Zbudować lekki model CNN, który na podstawie samego obrazu `256x256` rozpoznaje, do jakiego stanu obraz został wcześniej zdownscalingowany.

Pytanie badawcze: czy po finalnym obrazie można rozpoznać ślad wcześniejszego zmniejszenia skali?

Stan	Znaczenie	Klasa
CTRL	obraz kontrolny, bez degradacji	0
x8	downscale z czynnikiem 8	1
x16	downscale z czynnikiem 16	2
x32	downscale z czynnikiem 32	3

Kontekst naukowy

Inspiracja: **Blind Image Quality Assessment Using Convolutional Neural Networks**

M. Frackiewicz, H. Palus, W. Trojanowski, Politechnika Śląska.

Autorzy pokazują, że prostszy CNN może być praktyczną alternatywą dla ciężkich metod BIQA, jeśli połączy się go z nowoczesną optymalizacją.

Co bierzemy z pracy

- lekka architektura konwolucyjna,
- lokalna normalizacja wejścia,
- uczenie na patchach,
- Optuna / optymalizacja bayesowska,
- Adam jako optymalizator.

Co zmieniamy

- zamiast oceny jakości: klasyfikacja,
- zamiast benchmarków TID2013/KADID-10k: próbki z RAISE,
- zamiast regresji: softmax na `CTRL/x8/x16/x32`.

Zachowujemy ideę: model ma być lekki, skalowalny i możliwy do trenowania bez bardzo rozbudowanej architektury.

Dataset RAISE

RAISE jest bazą naturalnych fotografii w formacie RAW/NEF.

W projekcie pełni rolę źródła obrazów naturalnych, z których generowane są kontrolowane próbki degradacji.

Założenia:

- jedno zdjęcie źródłowe nie może przeciekać między splitami,
- najpierw ustalany jest split źródła,
- dopiero potem generowane są warianty próbek dla tego źródła.

Aktualny status

Pipeline generowania danych działa end-to-end.

Obecny przebieg roboczy:

- przetworzono około 1000 obrazów źródłowych,
- utworzono 15000 próbek PNG,
- zapisano manifest.jsonl, splits.json i pipeline_config.json,
- dane są gotowe do dalszego treningu klasyfikatora.

Główne ograniczenie na dziś: czas przetwarzania i dostęp do GPU.

Skala danych

Aktualna konfiguracja:

- 5 par degradacji,
- 3 skale: 8 , 16 , 32 ,
- $5 \times 3 = 15$ próbek z jednego obrazu źródłowego.

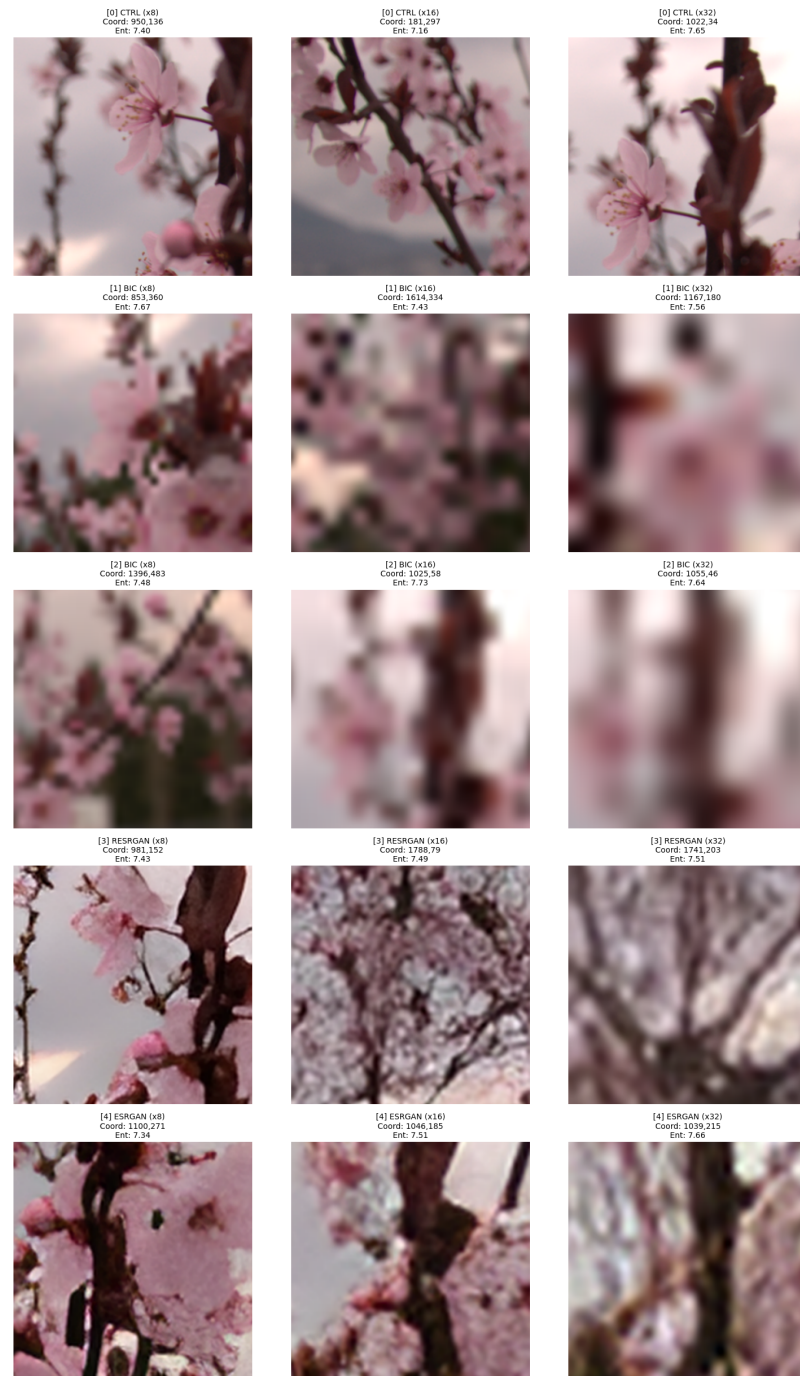
Dlatego:

1000 obrazów źródłowych $\times$ 15 próbek = 15000 próbek

Pipeline

Przepływ operacyjny:

1. Google Drive trzyma wejściowe RAW/NEF.
2. Colab kopiuje batch do lokalnego `content`.
3. Pipeline generuje PNG 256x256 i manifest.
4. Wyniki są pakowane do ZIP.
5. ZIP wraca na Drive.
6. Dalszy trening powinien przejść na mocniejszy serwer z GPU.



Smart Crop

Smart crop wybiera fragment obrazu, który ma wystarczająco dużo informacji dla klasyfikatora.

Mechanizm:

- losowanie 16 kandydatów,
- ocena entropią Shannona,
- filtr wariancji Laplaciana,
- fallback do centrum, jeśli żaden kandydat nie spełni progów.

Cel praktyczny: unikać płaskich fragmentów, które nie niosą widocznego śladu degradacji.

Warianty danych

Każdy obraz źródłowy przechodzi przez pięć par wariantów dla trzech skal `x8` , `x16` , `x32` .

Pair	Downscale	Upscale	Rola
0	CTRL	CTRL	kontrola
1	NN	BIC	aliasing / geometria
2	BOX	BIC	rozmycie
3	BIC	Real-ESRGAN	rekonstrukcja AI
4	LANCZOS	ESRGAN classic	wygląd vs wierność

Etykieta klasy zależy od stanu/skali, nie od pary degradacji.

Manifest jako centrum projektu

Pipeline zapisuje:

```
1K_out/  
├── train/  
├── val/  
├── test/  
├── stress_test/  
├── manifest.jsonl  
├── splits.json  
└── pipeline_config.json
```

`manifest.jsonl` służy do odtworzenia:

- pochodzenia próbki,
- splitu,
- pary degradacji,
- skali downscalingu,
- lokalizacji pliku PNG,
- etykiety klasy dla modelu.

Rekord manifestu

```
{
  "sample_id": "..._p3_s16",
  "source_image_id": "...",
  "source_file": "r000da54ft.NEF",
  "split": "train",
  "is_stress_test": false,
  "pair_id": 3,
  "degradation": {
    "downscale": {"name": "Bicubic", "abbr": "BIC"},
    "upscale": {"name": "Real-ESRGAN", "abbr": "RESRGAN"},
    "scale_factor": 16
  },
  "crop": {
    "size_px": 256,
    "top_left_xy": [1788, 79],
    "fallback_used": false
  },
  "file": {
    "path_relative": "train/...png",
    "format": "PNG"
  }
}
```

Model CNN

Adaptacja idei BIQA

- wejście: PNG `256x256x3`,
- lokalna normalizacja `3x3`,
- `tf.data` ładuje obrazy batchami,
- ten sam duch: prosty CNN + nowoczesne strojenie.

Główna zmiana: głowica nie przewiduje jakości obrazu, tylko stan downscalingu.

Architektura

- bloki `Conv2D -> BatchNorm -> MaxPool`,
- filtry: `32, 64, 128, 256, 256, 256`,
- warstwa gęsta strojona przez Optunę,
- `Dropout` i Adam,
- `Dense(4, softmax)`.

0

`CTRL`

brak degradacji

1

`scale=8`

downscale x8

2

`scale=16`

downscale x16

3

`scale=32`

downscale x32

Ograniczenia obliczeniowe

Colab Free okazał się niewystarczający już na etapie strojenia hiperparametrów.

Problemy:

- wolne generowanie danych dla większych batchy,
- kosztowne modele ESRGAN,
- limit czasu sesji,
- ograniczona stabilność GPU,
- długi trening z Optuną i walidacją.

Wniosek: pełne przetwarzanie RAISE i docelowy trening wymagają mocniejszego serwera z GPU.

Następne kroki

1. Dokończyć przetwarzanie pełnego datasetu RAISE.
2. Uruchomić trening klasyfikatora na serwerze z GPU.
3. Ocenić accuracy globalnie oraz per `scale_factor` .
4. Sprawdzić, które pary degradacji najłatwiej lub najtrudniej ukrywają ślad downscalingu.
5. Przygotować końcową ewaluację na `stress_test` .